

# Pose Estimation and Digit Recognition for Automated IC Chip Testing

Adrian Iannantuono, Jonathan Horton, Amrit Kaur  
University of Ottawa

**Abstract** – This paper outlines a project utilizing machine vision to detect incorrectly placed integrated circuit microchips in a test socket. The project will also cover an automated digit recognition system to integrate and accelerate the existing testing process. The pose estimation section of the project utilizes classical machine vision algorithms, not relying on any machine learning or neural network approaches. The digit recognition section utilizes classical algorithms for digit locating and segmentation as well as machine learning to obtain the final digit result which is not the focus of this project. Results from the implementation of this project show an accurate and replicable determination of chip pose and serial number extraction. The proposed solution shows it is able to perform as expected on various test images but further testing is needed to ensure it performs well more scenarios with varying chip orientation and lighting.

**Keywords** – Pose estimation, Digit recognition, Automated testing

## INTRODUCTION

The ATOM Handler is a machine used in many silicon test laboratories to automate pre-production volume testing of IC test chips. The handler works by using a small suction vacuum mounted on a 3D gantry system to pick up test chips off trays and load them into the testing socket. The handler allows for testing of up to 72 devices at a time. Once a test chip is loaded into the test socket, the plunger head comes down on to the test chip (Figure 2) and applies pressure to ensure good contact between the test chip pins and the socket pins.

While the handler is sequentially testing devices, issues may occur when it comes to test chip placement in the test socket. If the device is placed incorrectly, the pressure from the plunger can be damaging to test chip pins and at times it may cause the handler to interrupt testing. This issue is costly on time and materials.

The test chips placed on trays and loaded into the handler are hand serialized by lab technicians. The serial numbers are important and used to label the test data being collected. Before starting the handler, engineers must enter all serial numbers to a .txt file. This process is time consuming, especially when testing many devices.

The goal of this project is to develop algorithms to fully automate the detection of incorrectly placed test chips and the reading of serial numbers.

Note: images of test chips have been censored as they contain confidential information



Figure 1. Atom Handler Setup in Lab

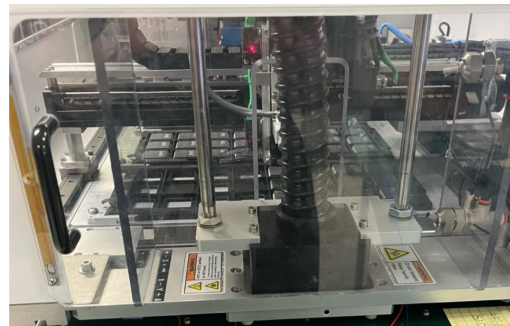


Figure 2. Plunger Head Lowered on Test Socket During Testing, Test Chips on Loaded Trays in the Back

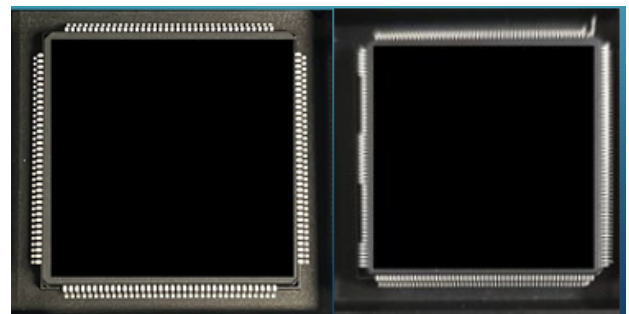


Figure 3. Test Chips with Good Pins (left) and Damaged Pins (Right)

Due to the space constraints within the handler and the location of the plunger, placing the camera directly above the test socket is not viable. Therefore, the camera must be placed off to the side, with a view of the test chip. The proposed set up is shown in Figure 4.

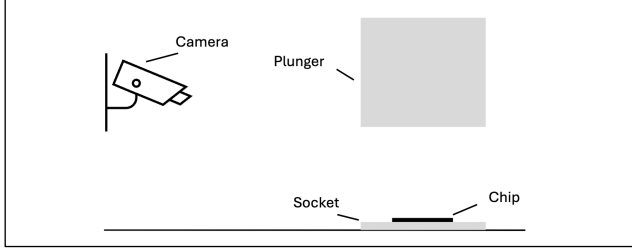


Figure 4. Camera Placement for Pose Estimation Proposed Solution

## RELATED WORK

The semiconductor industry uses computer vision for a variety of applications such as: lithography and pattern recognition, wafer inspection/defect detection; wafer/die sorting and process automation. Each vision system is custom tailored to the application in the semiconductor IP development and manufacturing industries.

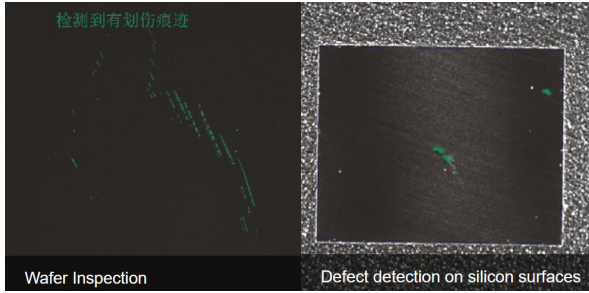


Figure 5. (left) Wafer Surface Scratch Inspection [1]. Figure 6. (right) Detection of edge/corner damages, cracks and fragments on silicon surfaces [1]

There are companies in the semiconductor industry which produce vision systems for pick and place applications such as that proposed in this paper. These vision systems are well calibrated and use equipment tuned for micro vision applications. Due to the secretive and competitive nature of the industry, the research behind these systems is not openly available.

Thus, the pose-estimation methodologies implemented in this paper rely mostly on classical pose estimation and edge detection algorithms tuned to work with the available cameras with limited budget. Such classical methods include: homography matrix calculation, Canny Edge detection and Hough transforms. The applications of these algorithms and the research behind them is detailed in the next section.

For the digit recognition portion of the solution, there are a lot more openly available resource. For example, DIGITNET is a deep learning algorithm used to recognize handwritten digits in historical data sets. In terms of benchmarking, several datasets exist, the most common of which is the MNIST dataset which was first compiled in the 1990s. Generally, most applications compress the digits to an array of 28 by 28 pixels before processing. [2]

Seeing as the digit recognition algorithms are open access, they can easily be utilized in a variety of applications; they can also be used to create precise models based on handwriting samples.

The combined solutions proposed in this paper allow for a robust and budget friendly implementation of a machine vision system in a small silicon test lab.

## METHODOLOGY

### A. Pose-Estimation

The proposed solution block diagram is shown in Figure 5 below. The process involves capturing images of the test chip in the test socket and cropping the image to focus on the test chip. From there, a perspective correction is implemented so that the chip can be seen “from above.” Once these pre-processing steps are completed, the test chip must be isolated from its surroundings by flood filling to create a binary mask. Finally, Canny Edge detection is used to find the edges of the test chip and Hough Transform is used to determine if the chip has been placed correctly or not.

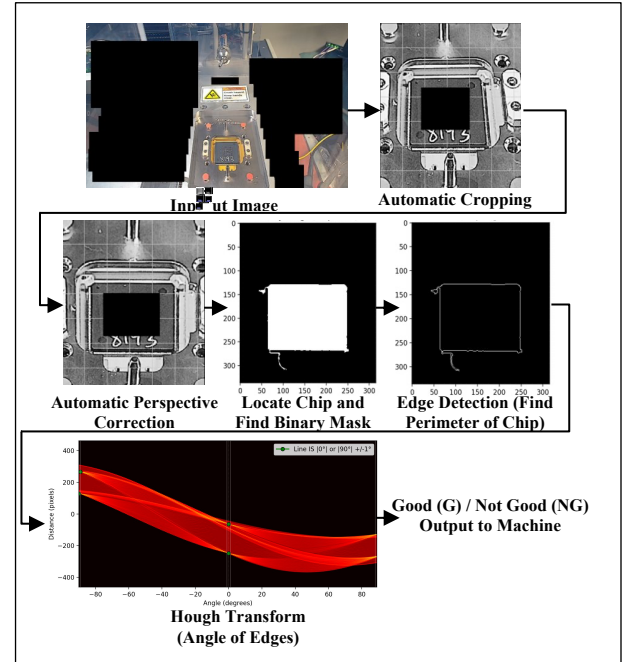


Figure 7. Pose-Estimation Proposed Solution Block Diagram

The input image shown in Figure 7 depicts a very wide angle of the scene. To ensure accuracy for the chip placement algorithm, the image must be cropped to the region of interest. To ensure precise cropping, red dots were placed on top of screws that connect the metal docking plate to the handler. The docking plate and related components are CNC milled components with  $\pm 0.3\text{mm}$  tolerance. With this method, accurate cropping is ensured even if camera placement moves slightly.

For the algorithm to find these red dots, an HSV (Hue, Saturation, Value) mask is applied to the image to isolate red regions defined by upper and lower red hues. With such a wide camera angle, other red regions in the photo can also be detected. To mitigate this undesirable effect, a ROI mask (Region of Interest) is created. The ROI mask defines which regions of the photo can be ignored. Both masks are applied to the image, and the manually placed red dots can be detected and isolated. To ensure the same point is not detected twice (leading to incorrect cropping and perspective) connected component analysis is used to identify connected pixels in the masked binary image. The two nearby detected point, if they exist, are merged to one.

The centroids of the four red regions are unpacked into x and y coordinates. The x and y coordinates are used to first crop the image to focus on only the region containing the test chip.

A Homography matrix calculation is done to correct the perspective [3]. A Homography is a 3x3 transformation matrix that maps points from one plane to another in a perspective transformation. The x any y coordinates of the centroids define the “destination points,” the rectangular region that each pixel of the image must be mapped to. Figure 8 demonstrates the intended outcome of using a Homography matrix to correct the camera’s slanted perspective.

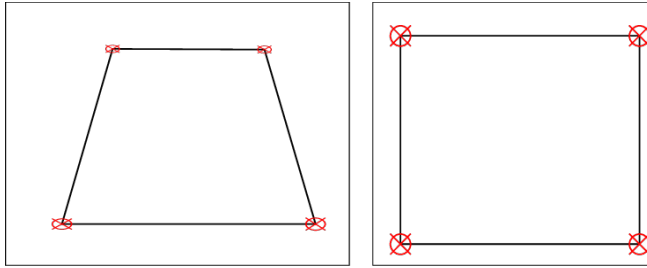


Figure 8. Intended Outcome of Homography Matrix Perspective Correction [4]

The Homography Matrix allows for the following transformation:

$$[x', y', w'] = H \cdot [x, y, 1]^T$$

Where:

$H$  is the Homography Matrix

$(x, y)$  are points in the source image

$(x', y') = (\frac{x'}{w'}, \frac{y'}{w'})$  are the corresponding points in the destination image

The matrix is calculated using RANSAC: Random Sample Consensus (RANSAC) to robustly estimate the best Homography by ignoring outlier points that may not fit the transformation model.

Warp Perspective (OpenCV-Python) is used to apply the Homography matrix to the original image. Warp Perspective maps each pixel in the destination image back to the corresponding location in the original image using the inverse

of the Homography matrix. The pixel value is interpolated from the original image.

With the perspective corrected, the area of the test chip can be isolated from the background. Flood filling is used to segment and identify the largest connected object in the binary mask. Flood filling is applied starting from point 0, 0 (top left of cropped image) to fill any holes inside the mask. Based on the camera setup, it is known that the top left start point is always the socket, and not the actual test chip

After flood filling, the connected background region is white. To isolate the foreground object, the inverted image is computed. The flood-filled mask is then combined with the original binary mask to fill gaps within objects. This step ensures that the mask includes only the largest connected object, removing any unwanted holes or small disconnected regions.

Once the system has a binary mask of the chip, it performs edge detection to get a clean output of the chips perimeter. This is done so that later processing steps can locate this edge and determine if the edge is straight in the image. This step is important in pose estimation because an accurate and thin line outline of the chip is needed for the Hough Transform algorithm. Four edge detection algorithms were tested and evaluated based on their ability to extract a complete outline of the chip. The algorithms tested include Sobel, Prewitt, Laplacian of Gaussian (LoG) and Canny. The Sobel edge detection algorithm outlined in [5] uses the gradient in the x and y directions which makes it biased to edges that are 0° or 90°. The Prewitt edge detection algorithm defined in [6] suffers from the same tendency to bias towards increased detection of vertical and horizontal lines. The LoG algorithm developed in [7] utilizes an isotropic kernel which allows it to be equally sensitive to edges in all directions. The Canny algorithm calculates the first order derivative to locate edges in all directions while adding additional processing steps that locates and connects strong and weak edges to produce a more complete image [8].

The last step in the algorithm is to use the edge detected image to determine if the sides of the chip are aligned with the holder. Since it is known that the camera is centered horizontally in relation to the chip and the image has been corrected in the perspective correction step, it follows that all sides of the chip should be very close to 0° (vertical) or 90° (horizontal) relative to the camera. Although this assumption isn't perfect as the image may not be perfectly corrected in the image correction step or the camera may not be exactly centered, this assumption will prove to be reasonable given the constraints faced and the results obtained. These assumptions, after correcting the image, allow for the direct use of algorithms to determine the orientation of the lines in the edge detection image to determine if the chip is sitting correctly. The approach to determine line orientation that will be used in this paper is Hough Transform. Hough Transform is a technique used to identify straight lines in an edge detected image developed by [9]. It works by first mapping every pixel

in Cartesian space to a Polar space function where  $\theta$  is a variable in the following equation:

$$\rho = x_p \cos \theta + y_p \sin \theta \quad (1)$$

In (1) from [9],  $\rho$  represents the length of the line normal to the edge that passes through the origin. The terms  $x_p$  and  $y_p$  represent the x and y Cartesian coordinates of the pixel and remain constant.  $\theta$  represents the angle of the line in relation to the x axis. Once the pixel coordinates are defined in polar space, the algorithm uses (1) to graph the defined function for each pixel, varying  $\theta$  for all possible angles ( $-90^\circ$  to  $+90^\circ$ ). This produces a periodic function for each  $(x_p, y_p)$  in the edge detected image which gets plotted on the same graph. Finding the intersection of these summated periodic functions gives an angle and position of the detected line in the image. An example of this can be seen in Figure 9 where the points B, N, P and A form a virtual line in the original image. Graphing the periodic function and summing these functions produce an intersection point that defines the angle and pixel location of the line that is formed by passing through the defined points.

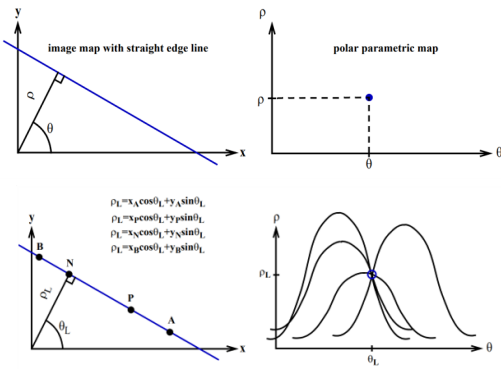


Figure 9. Hough Transform Definition Example [10]

Performing the Hough Transform on the edge detected image of the IC sitting in the holder will output the orientation and location of the four sides of the IC. Since these periodic functions are overlaid and summed, there will be regions of greater intensity and lower intensity. These represent the lengths of the lines found in the image. Finding the most intense regions (peaks) of the generated Hough Transform data allows for the locating of the four sides of the IC chip as these sides define the perimeter of the chip and the assumption is made that the edge detected image doesn't have a significant amount data other than the pixels representing the perimeter of the chip. Due to this assumption, the four strongest peaks of the Hough Transform data are extracted and represents the angle and position the four perimeter lengths of the chip. A direct analysis of these perimeter lengths is then done to determine if the angle of each of these sides are within a specified tolerance angle. For a good sitting part, these four lines should be very close to  $|0^\circ|$  (vertical) or  $|90^\circ|$  (horizontal) relative to the camera. The tolerance angle was chosen

experimentally as  $1^\circ$  which offers a good reliability as well as robustness to errors in the perspective correction step which could produce a distorted image. This means that any line that is not within  $|1^\circ|$  of  $|0^\circ|$  (vertical) or  $|90^\circ|$  (horizontal) is flagged and considered not aligned. If any of the four sides of the chip are found to not be aligned, the system will output that the chip is not sitting correctly in a Not Good (NG) condition. The requirement for all four sides to have a Good (G) condition offers additional robustness to the system by making it able to stop the machine from proceeding with the test even if a good outline of the chip cannot be acquired for all sides of the chip. The system will also display the four detected sides of the chip in green or red depending on if it meets the angle requirement.

## B. Digit Recognition

For this project, there are two scenarios in which number recognition can be applied. To maintain flexibility, a pre-processing technique for extracting each digit was developed for scenarios where some noise is introduced into the image and where no noise is introduced through manual cropping. These two scenarios are illustrated in Figure 10. The first shows a chip placed in the socket, with the image manually cropped to find the exact pixel values delimiting the chip's face. The second image shows a top-down view of the chip tray.

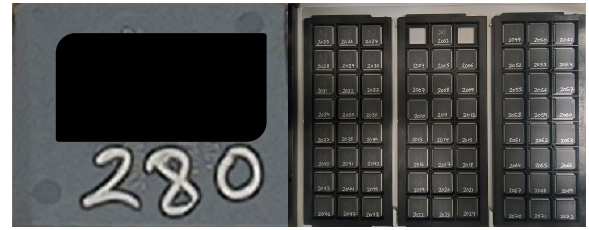


Figure 10. Chip in Socket (Left) and Chip Tray (Right)

In the context of digit recognition, noise is defined as anything introduced at the edge of the image, particularly the pins. This represents scenarios where a chip in the chip tray has slightly shifted, causing the manual crop to include the pins, or where the chip has been slightly misplaced in the tray. Given that the pins are silver and appear white on camera, they can introduce noise into the image that cannot simply be removed through thresholding.

To recognize digits in an image, a model needs to be trained. The MNIST dataset [11] was selected due to its status as a general standard benchmark in the field, though the goal of the project was not to develop a better digit recognition model. Highly precise models already exist, capable of recognizing digits with 99.7% accuracy, so potential improvements would be marginal. Instead, the goal was to develop a system that extracts each individual digit for input into the model.

The process for determining the contours of each digit is outlined in Figure 11 and additional examples can be seen in Appendix B. First, the image is cropped to contain the chip's

face, then converted to grayscale. Next, the image is converted to a binary mask. Otsu thresholding was initially applied, but the variable lighting conditions of the sample images caused excessive noise. However, Otsu thresholding performs effectively in the controlled lighting conditions of the plunger area. Thus, a variety of manual thresholds were applied, and 75% was selected as a compromise between maintaining text visibility and reducing noise. Next, contours of the remaining elements in white were extracted using OpenCV's `findContours()` function in `RETR_EXTERNAL` mode [12], which selects only the outermost contours of the digits. Next, it is assumed that the text is at least 5 and 10 pixels in width and height, respectively, to exclude small contours unlikely to represent a number.

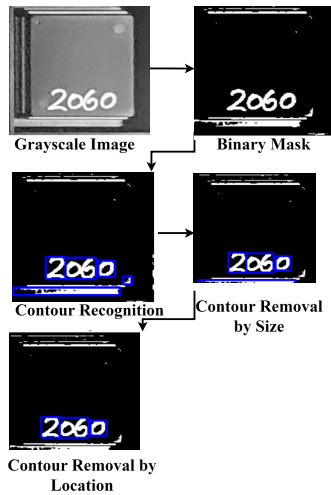


Figure 11. Process for Locating Contours of Digits

Fifthly, contours are removed based on their location according to three criteria:

1. To narrow down the number of contours to the four most likely representing digits, only the four elements that are radially closest to the center are kept.
2. To ensure vertical alignment, the variance of all possible subsets of three elements is calculated. For the element not included in the subset with the least variance, it is removed from the set if its vertical center differs by more than half the median height of the subset with the least variance. If it does not, the fourth element remains in the set.
3. As a final check, from left to right, if the horizontal distance between two sequential contours exceeds three-quarters of the median width of all contours, the leftmost contour of the pair is removed from the set.

After applying this process to all images, the remaining contours, representing digits, are fed into the model trained on the MNIST dataset. For placeholder purposes, a random forest

model was used to classify each remaining contour into a digit ranging from 0 to 9. Since the MNIST dataset compresses images to 28 by 28 pixels, the contours are also resized accordingly. If a contour is smaller in either dimension, black pixels are added to pad the edges around the image. If it is larger, the image is compressed to meet the size requirement. Each image can then be labeled with every detected digit.

## RESULTS

### A. Pose-Estimation

With the limitations of the camera setup, the proposed solution yields good results, with room for improvement. The simplest and possibly most effective improvement would be to mount a smaller camera within the test area (as close to the plunger as possible, without compromising its functionality), to gain a better top-down perspective of the test chip.

The cropping and perspective corrections methods are robust and repeatable. However, the Homography matrix for perspective correction can be further improved by using a direct overhead shot of the test socket as the destination image.

The binary mask to detect the test chip is fairly accurate but has identifies small portions of the test socket as part of the test chip. The above-mentioned improvements would likely help with this issue, as the bottom edge of the test chip “touches” the edge of the socket from the current camera perspective. Since the flood filling mask searches for large, connected regions, a better view would provide more distinction between the test chip and test socket/docking plate.

Four edge detection algorithms were tested to determine which technique performs the best. The algorithms tested were Sobel, Prewitt, Log and Canny. The effectiveness of each algorithm was compared by visually inspecting the outputted outline of the chip and comparing this to the original image where the chip can be seen. Figure 12 shows a comparison between the tested edge detection algorithms with the detected edges overlayed on the perspective corrected image. From this image it can be seen that the Canny edge algorithm performs the best out of all the tested edge algorithms applied to the same sample image containing a binary image mask of the chip. Not only do the edges in the Canny image represent a more accurate outline of the chip, but the edges are also thinned nicely which will remove noise and provide a more accurate input for the Hough Transform step that comes next.

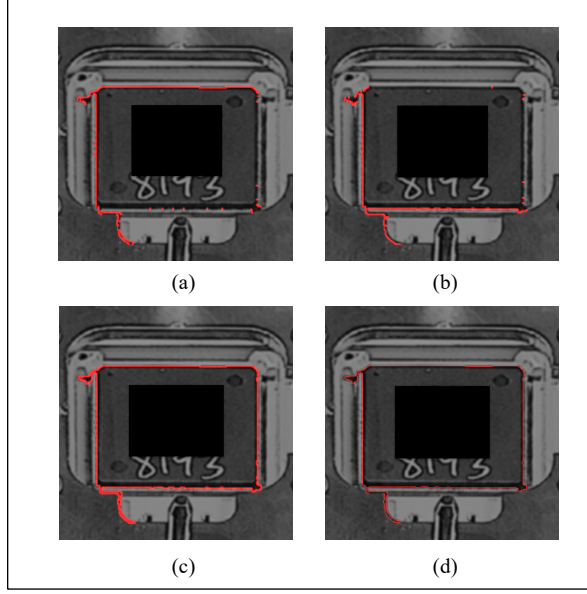


Figure 12. Comparison of Edge Detection Algorithms Overlayed on Corrected Image. (a) Sobel, (b) Prewitt, (c) LoG, (d) Canny

The edge detected with perspective correction Canny image of the IC chip is then fed into a Hough transform which computes and locates the direction and position of all line candidates in the image frame. Following the processing steps defined earlier, the proposed system locates the four perimeter lengths of the chip and determines if each of these sides are within  $1^\circ$  of being horizontal or vertical. Seen in Figure 13, the system then outputs a graph displaying all line candidates with the four perimeter lines located with either a red or green dot. The dot is green if the line is within the specified tolerance and it is red if it is outside of the specified tolerance. A visual is also displayed of the detected perimeter lines (in either green or red dependent on its derived category) overlayed on the Canny edge detected image. This allows for the user to visually confirm that the system has correctly classified the chips orientation and is achieving the expected results. Figures 13(a) and 13(b) show results for a Good (G) sitting chip which was detected as Good (G). Figures 13(c) and 13(d) show results for a Not Good (NG) sitting chip which was detected as Not Good (NG). From this it can be seen that the two vertical sides of the chip were detected to be outside of the tolerance, therefore making the system output a Not Good (NG) signal.

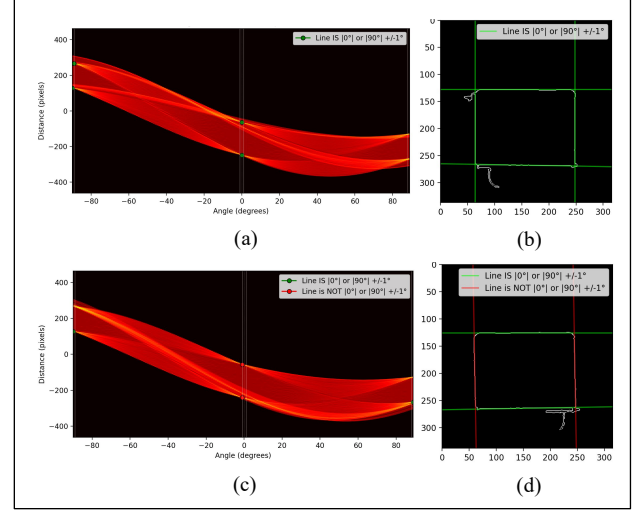


Figure 13. Four Perimeter Lines Angles Shown With  $\pm 1^\circ$  Tolerance Lines, (a) Hough Transform graph for Good sitting part, (b) Perimeter lines overlayed on Good sitting part, (c) Hough Transform graph for Bad sitting part, (d) Perimeter lines overlayed on Bad sit

Testing of the algorithm was completed using X well-lit test images of the IC chip sitting in various orientations in the test socket. The orientation and visual system output for each case can be seen in Appendix A. A summary of the results can be seen in Table 1 where the actual orientation refers to the known condition on whether the chip is placed correctly. This condition was confirmed when parts were placed properly by lowering the gantry system and performing a test of the chip. It was assumed that any visually skewed chip in relation to the test socket was considered a Not Good (NG) placed chip. Confirmation that a badly placed part will result in failure of the testing process or damage to the equipment was not done as to not damage the sensitive equipment and IC chips. Although this assumption was made, machine operators confirmed that the test process fails when parts are placed in similar orientations as tested in this report.

Table 1. Implemented Solution Pose Estimation Results on Various Test Images

| Image Number | Actual Orientation<br>(G=Good, NG=Not Good) | Implemented Solution Detected Orientation<br>(G=Good, NG=Not Good) |
|--------------|---------------------------------------------|--------------------------------------------------------------------|
| 1            | G                                           | G                                                                  |
| 2            | NG                                          | NG                                                                 |
| 3            | NG                                          | NG                                                                 |
| 4            | NG                                          | NG                                                                 |
| 5            | NG                                          | NG                                                                 |
| 6            | NG                                          | NG                                                                 |
| 7            | NG                                          | NG                                                                 |
| 8            | NG                                          | NG                                                                 |

## B. Digit Recognition

As illustrated in Figure 15, there are several issues associated with the current approach to cropping numbers. For instance, if a digit is too close to the edge, the pins may be included in the contour of the number, introducing additional noise that complicates the model's ability to accurately read the digit. Additionally, overlapping digits can occur when two numbers are written closely together. Although tools like DIGITNET could potentially recognize overlapping digits, this approach was not applied in the context of this project. The last two images in Figure 14 highlight the various types of unwanted noise introduced by poor lighting conditions. The leftmost image shows a scenario where part of the number adjacent to the "2" has been included along with its contour. Upon compression, the two contours appear separate, but prior to compression, they are touching. The last image depicts a "2" that has been poorly thresholded, resulting in black noise on its surface.

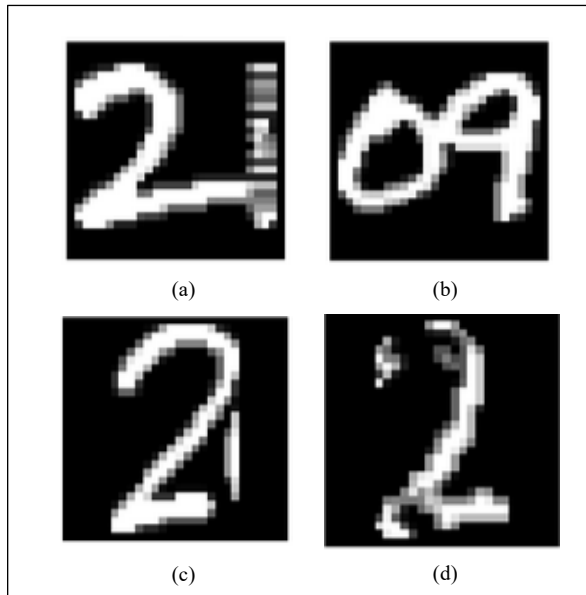


Figure 14. Comparison of Issues Arising in Digit Recognition (a) Pins on Right Side (b) Two Touching Numbers, (c) Unwanted Noise, (d) Poor Thresholding

Figure 15 showcases 100 results of the contour selection technique outlined in this paper. Of these, 86 were correctly classified, indicating a fair degree of accuracy. Results could be improved by creating a dataset trained on the handwriting of the technicians who write the digits on the chips. Additionally, some numbers are poorly thresholded due to manual thresholding, introducing black noise on the surface of these digits. Lastly, the placeholder model used in this project misclassifies some of the provided images, such as "7" for "2" and "4" for "9," and vice versa.

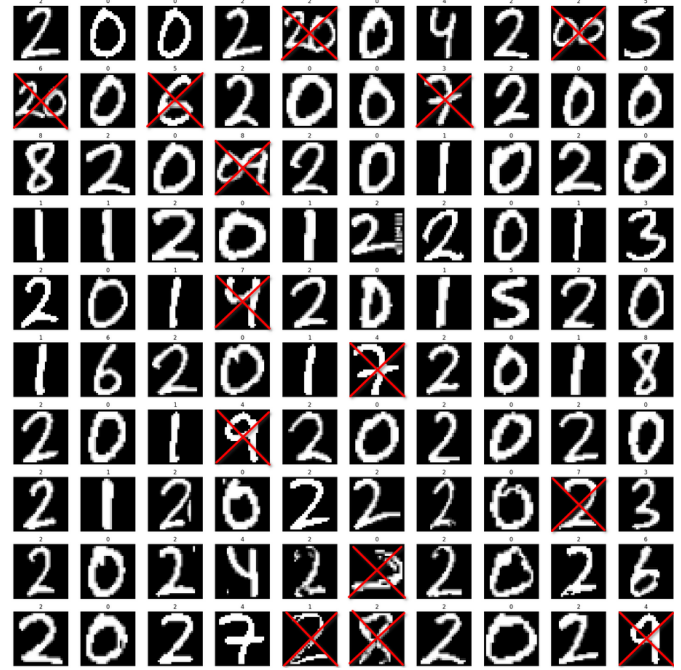


Figure 15. Overview of the Classification of 100 Contours Red "X" Indicates Misclassification

## DISCUSSION

### A. Pose-Estimation

The results of the pose estimation algorithm outlined in this paper show that the solution works effectively for all 8 test images. The system is able to successfully characterize the chips orientation in the test socket and output a Good (G) or Not Good (NG) signal to the testing machine to stop testing for all test images.

Although the implemented pose estimation solution outlined in this paper performed well and produced consistent and replicable results with the images tested, further testing is needed to confirm the effectiveness of the solution with varying lighting conditions. While developing the solution outlined in this paper it was found that lighting conditions had a major impact on its ability to recognize and segment the IC chip in a binary mask. As seen in Figures A.4, A.5 and A.8, the lack of contrast between the top edge of the chip and the test socket caused the system to not be able to obtain an outline of the complete perimeter of the chip. On this note, more testing on ideal lighting setups would be needed to improve the accuracy of the solution. The images taken with the webcam in this project were not calibrated. The perspective correction was done using the four red positioning dots which were placed on accurately machined markers on the testing plate. Improvements could be made to the accuracy of this project by calibrating the camera by taking test images with a checkerboard pattern placed on the testing plate surface. This calibration step would correct for lens and projection distortions and produce a more accurate measure of IC chip pose. Using a camera with a higher focal length that can fill the image sensor with more useful pixel data of the chip would

also be beneficial in order to improve the accuracy of the solution.

## B. Digit Recognition

The digit recognition algorithm presented in this paper effectively detects the contours of digits representing numbers, successfully classifying digits from 0 to 9. However, the system struggles to distinguish overlapping digits and occasionally misclassifies numbers.

To significantly improve results, several modifications can be implemented. Firstly, adopting a more accurate digit recognition model rather than the placeholder random forest model would likely reduce misclassifications. Additionally, developing a dataset comprising samples of technicians' handwriting would enhance classification accuracy by training the model on the unique idiosyncrasies of this specific handwriting style. Furthermore, installing a camera and LED strips in the tray area would ensure more consistent lighting conditions, thereby improving thresholding results. Employing a convolutional neural network tool like DIGITNET could also help separate overlapping digits within the model. Alternatively, informing technicians of the issue and encouraging them to ensure spacing between digits on future packages could mitigate the problem.

## CONCLUSION

In conclusion, the project successfully developed a solution to determine if an IC is correctly seated in the test socket. While the number recognition algorithm showed promising results, further refinement is needed to handle different lighting conditions and overlapping digits. The edge detection and pose estimation algorithms performed well, but improvements in lighting and camera calibration could enhance accuracy. The system's ability to classify the chip's orientation was effective, but additional testing is required to ensure robustness. Overall, the project demonstrated the potential of the proposed methods, with room for further optimization. Future work should focus on addressing the identified issues to improve the system's reliability and performance.

## REFERENCES

- [1] hongru, "Semiconductor\_Applications\_OPT Machine Vision\_SCI Machine Vision\_Lights\_Lenses\_Cameras," Optmv.net, 2024. [http://www.optmv.net/content/details150\\_646.html?v=Semiconductor](http://www.optmv.net/content/details150_646.html?v=Semiconductor) (accessed Dec. 06, 2024).
- [2] H. Kusatogullari, A. Yavariabdi, J. Hall, and N. Lavesson, "DIGITNET: A Deep Handwritten Digit Detection and Recognition Method Using a New Historical Handwritten Digit Dataset," *Big Data Research*, vol. 23, p. 100182, 2021. [Online]. Available: <https://doi.org/10.1016/j.bdr.2020.100182>. [Accessed: Dec. 06, 2024].
- [3] OpenCV, "Basic concepts of the homography explained with code" OpenCV. [Online]. Available: [https://docs.opencv.org/4.x/d9/dab/tutorial\\_homography.html](https://docs.opencv.org/4.x/d9/dab/tutorial_homography.html). [Accessed: Dec. 06, 2024].
- [4] J. zur Jacobsmühlen, S. Kleszczynski, G. Witt and D. Merhof, "Robustness analysis of imaging system for inspection of laser beam melting systems," *Proceedings of the 2014 IEEE Emerging Technology and Factory Automation (ETFA)*, Barcelona, Spain, 2014, pp. 1-4, doi: 10.1109/ETFA.2014.7005262. [Accessed: Dec. 06, 2024].
- [5] I. Sobel and G. Feldman, "A 3x3 isotropic gradient operator for image processing," presented at the Stanford Artificial Intelligence Project (SAIL), Rep. 671, 1968.
- [6] J. M. S. Prewitt, "Object enhancement and extraction," in *Picture Processing and Psychopictorics*, B. S. Lipkin and A. Rosenfeld, Eds. New York, NY, USA: Academic Press, 1970, pp. 75-149.
- [7] D. Marr and E. Hildreth, "Theory of edge detection," *Proceedings of the Royal Society of London. Series B. Biological Sciences*, vol. 207, no. 1167, pp. 187-217, 1980.
- [8] J. Canny, "A Computational Approach to Edge Detection," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679-698, Nov. 1986, doi: 10.1109/TPAMI.1986.4767851.
- [9] R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures", *Commun. ACM*, vol. 15, no. 1, pp. 11-15, Jan. 1972.
- [10] P. Payeur, Class Lecture, Topic: "Image Processing (Feature Detection)." ELG 5163, School of Electrical and Computer Engineering, University of Ottawa, Ottawa, 2024.
- [11] Y. LeCun, C. Cortes, and C. J. C. Burges, "THE MNIST DATABASE of handwritten digits," [Online]. Available: <https://yann.lecun.com/exdb/mnist/>. [Accessed: 11-Oct-2024].
- [12] OpenCV, "Structural Analysis and Shape Descriptors – Retrieval Modes" OpenCV. [Online]. Available: [https://docs.opencv.org/4.x/d3/dc0/group\\_imgproc\\_shape.html#gga819779b9857cc2f8601e6526a3a5bc71aa7adc6d6608609fd84650f71b954b981](https://docs.opencv.org/4.x/d3/dc0/group_imgproc_shape.html#gga819779b9857cc2f8601e6526a3a5bc71aa7adc6d6608609fd84650f71b954b981). [Accessed: Dec. 06, 2024].

## APPENDIX A – POSE ESTIMATION RESULTS

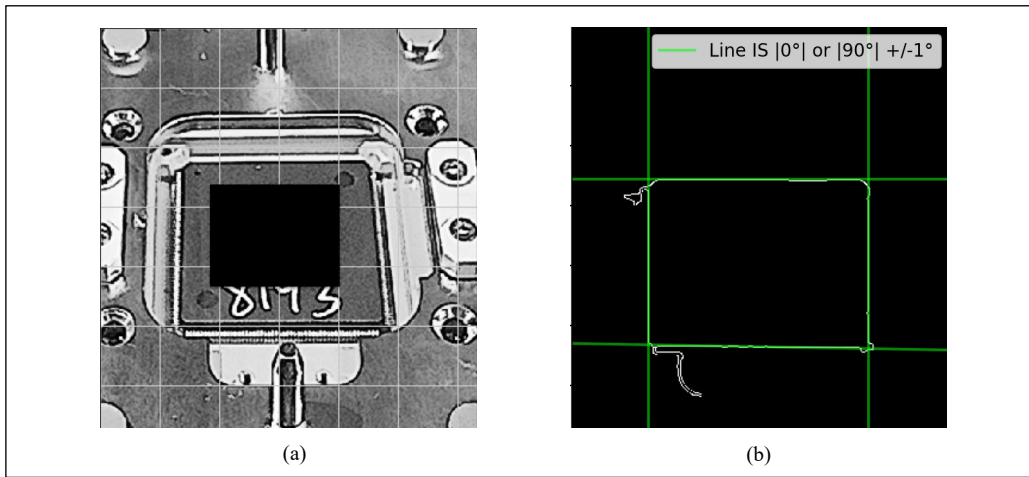


Figure A.1 Test Image #1 (Good Sitting Part). (a) Input (before perspective correction), (b) Output

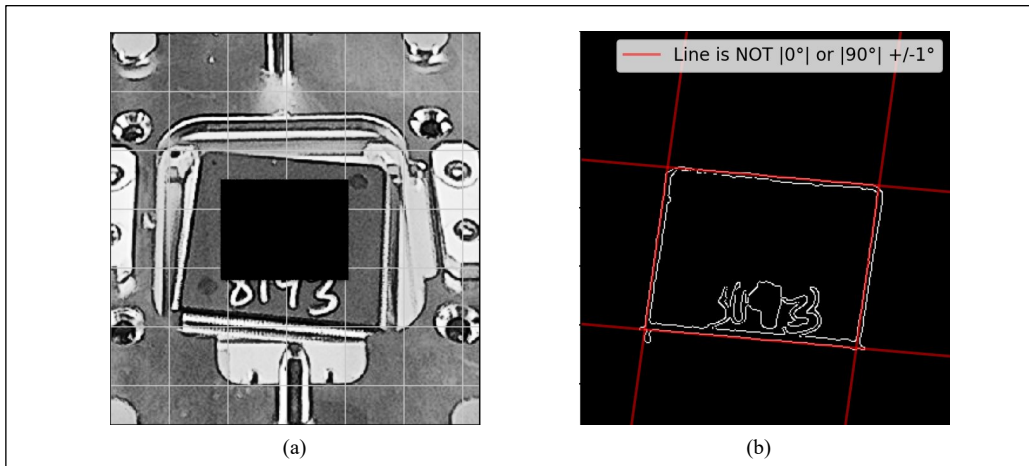


Figure A.2 Test Image #2 (Bad Sitting Part). (a) Input (before perspective correction), (b) Output

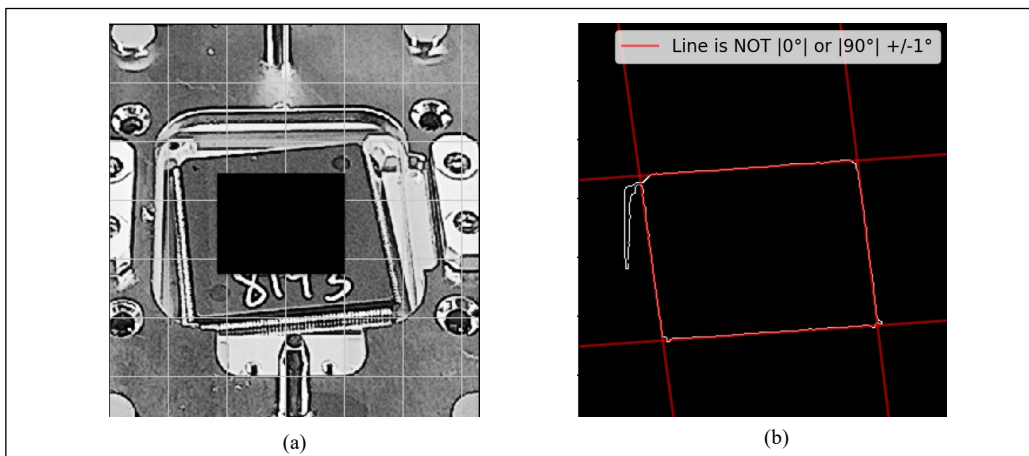
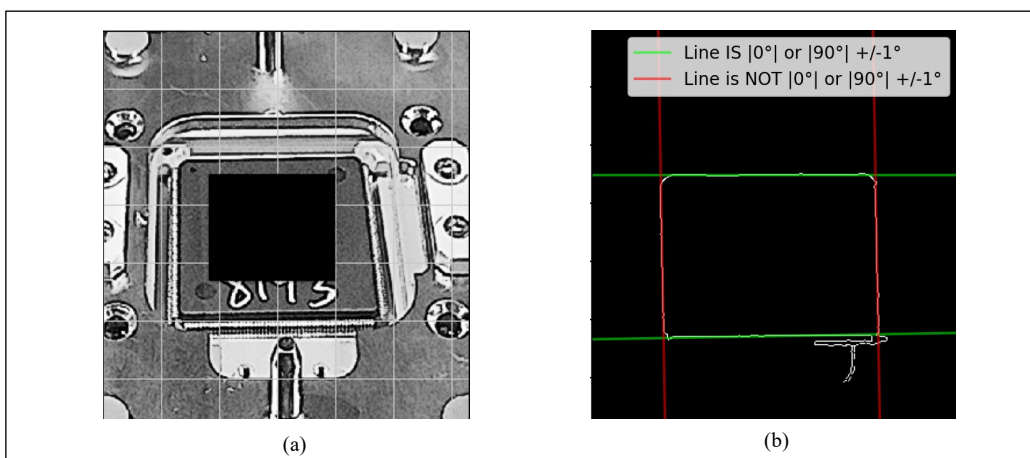
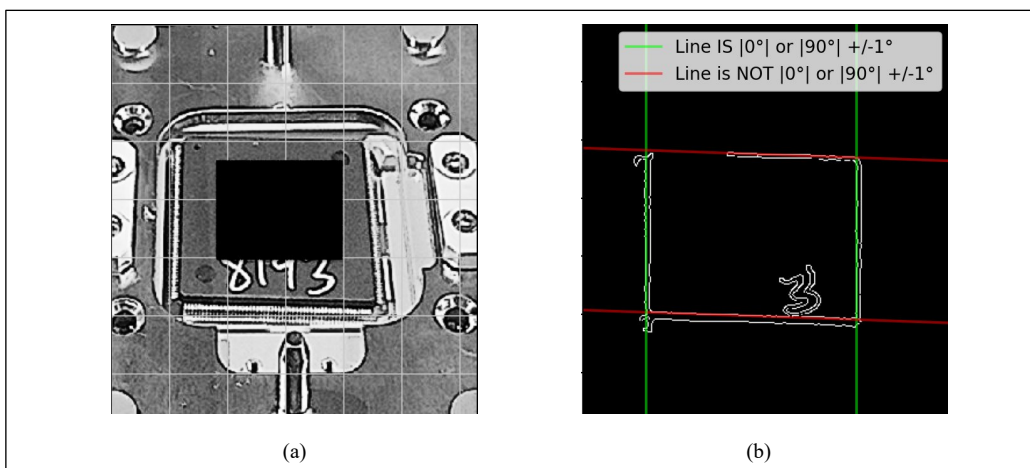
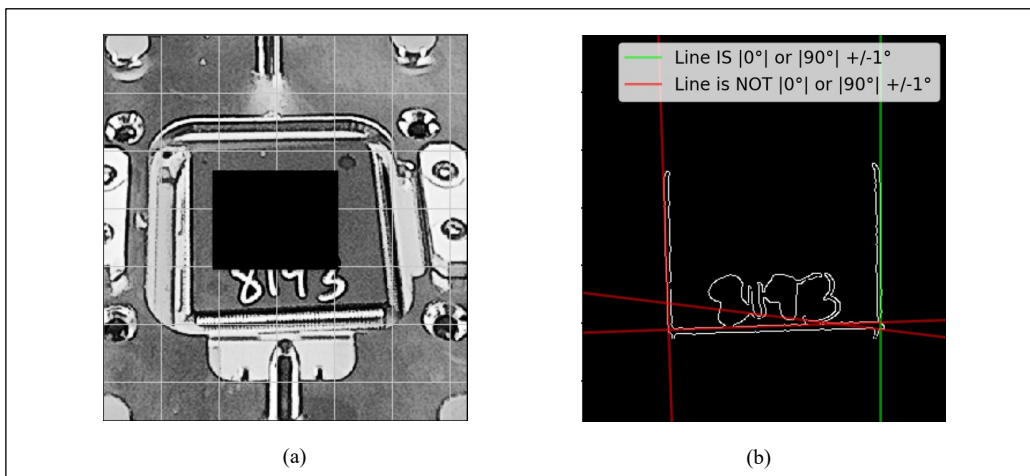


Figure A.3 Test Image #3 (Bad Sitting Part). (a) Input (before perspective correction), (b) Output



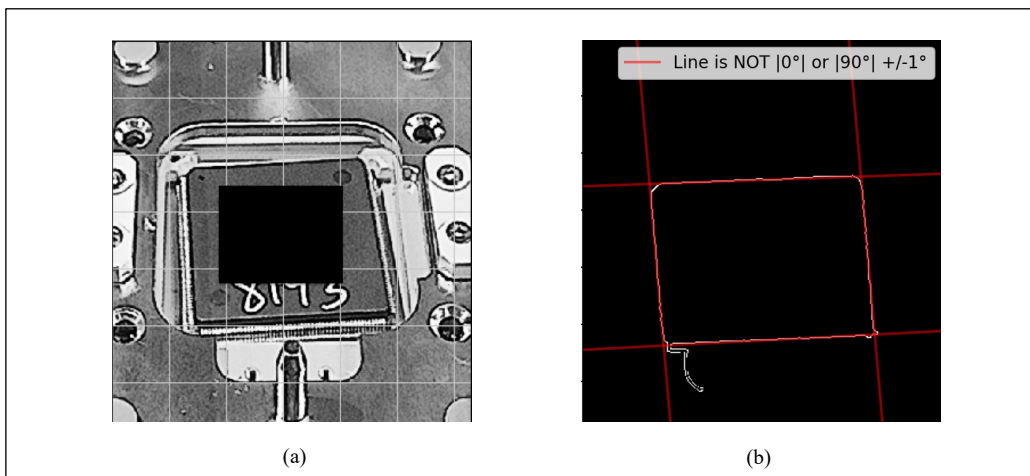


Figure A.7 Test Image #7 (Bad Sitting Part). (a) Input (before perspective correction), (b) Output

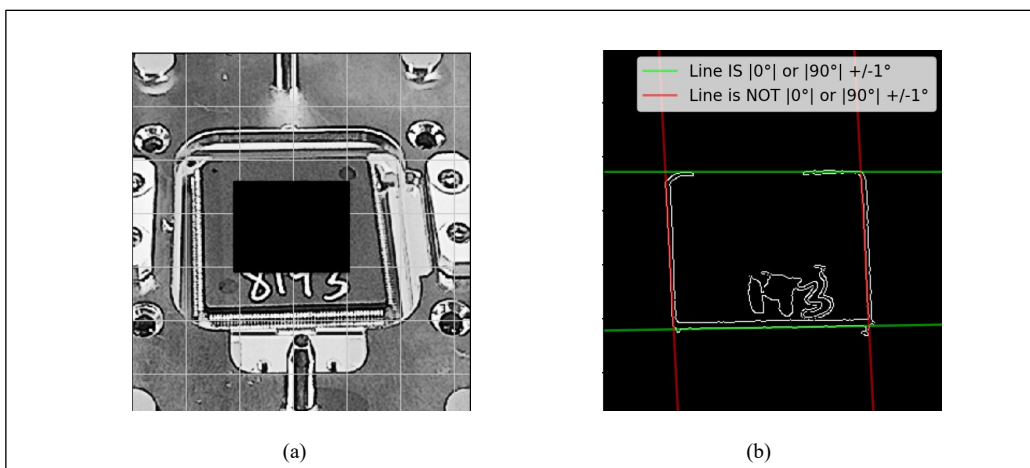


Figure A.8 Test Image #8 (Bad Sitting Part). (a) Input (before perspective correction), (b) Output

APPENDIX B – ADDITIONAL IMAGES FOR PROCESS OF ELIMINATING CONTOURS – JONATHAN HORTON

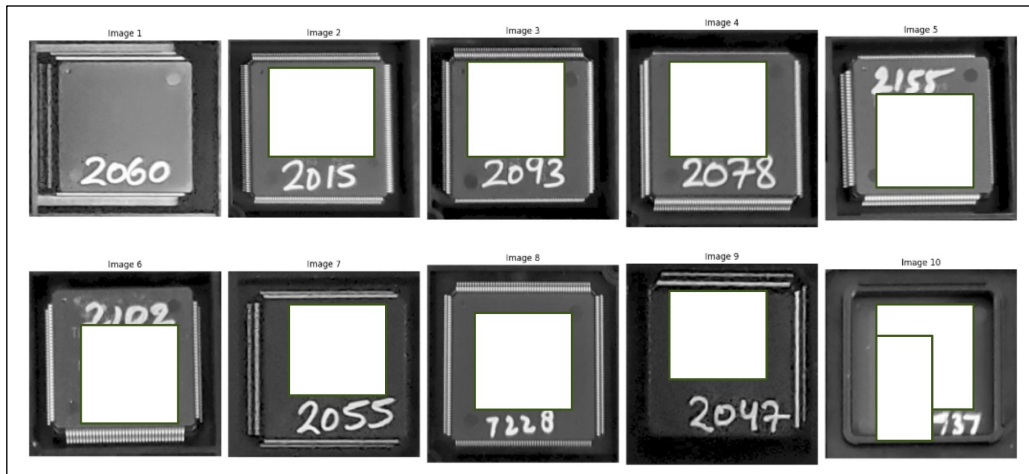


Figure B.1 Chip Tray in Gray Scale. Chip Information Has Been Occluded.

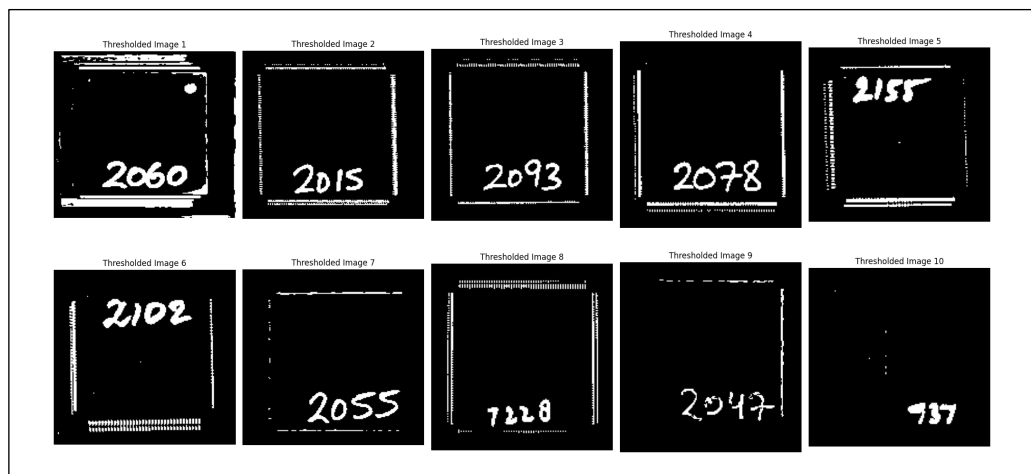


Figure B.2 Chip Tray With Manual Thresholding at 67%

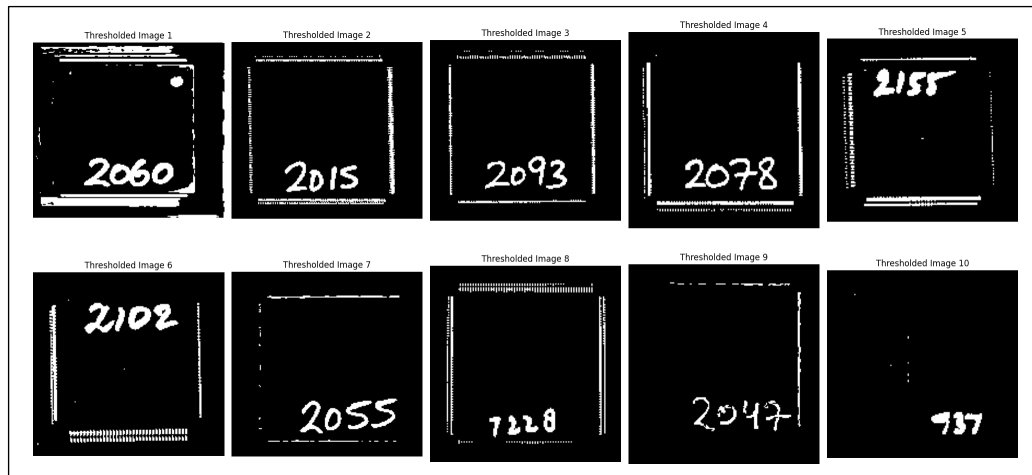


Figure B.3 Chip Tray with Manual Thresholding at 75%

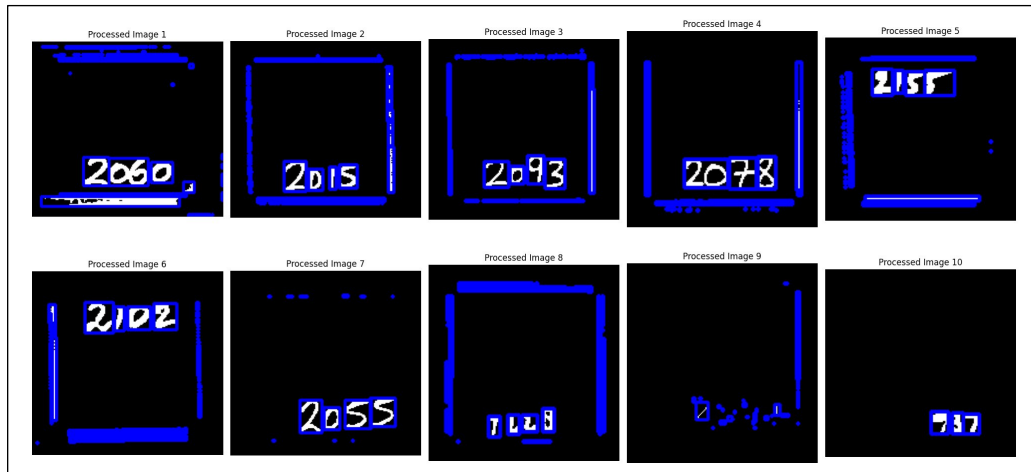


Figure B.4 Chip Tray With OpenCV's findContours(cv2.RETR\_EXTERNAL) Showing the Outer Most Contours

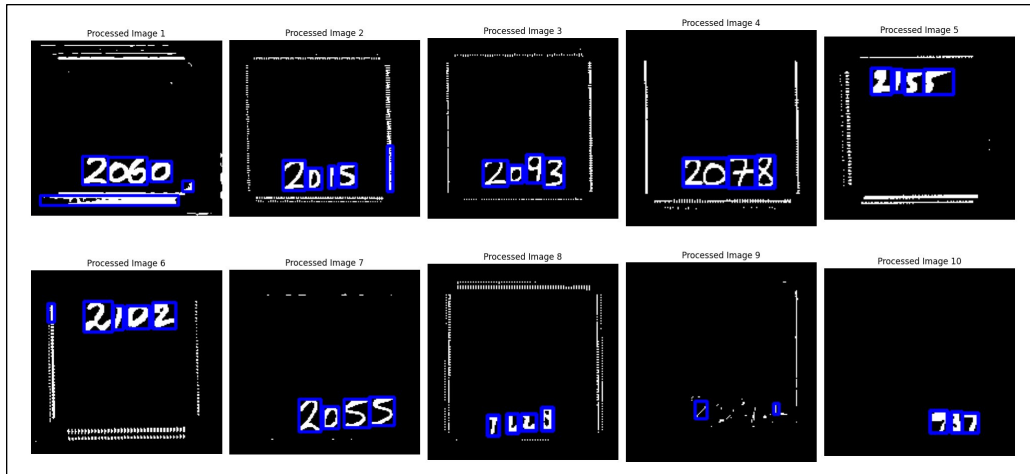


Figure B.5 Chip Tray with Contours Thresholded by the Size of Their Width and Height

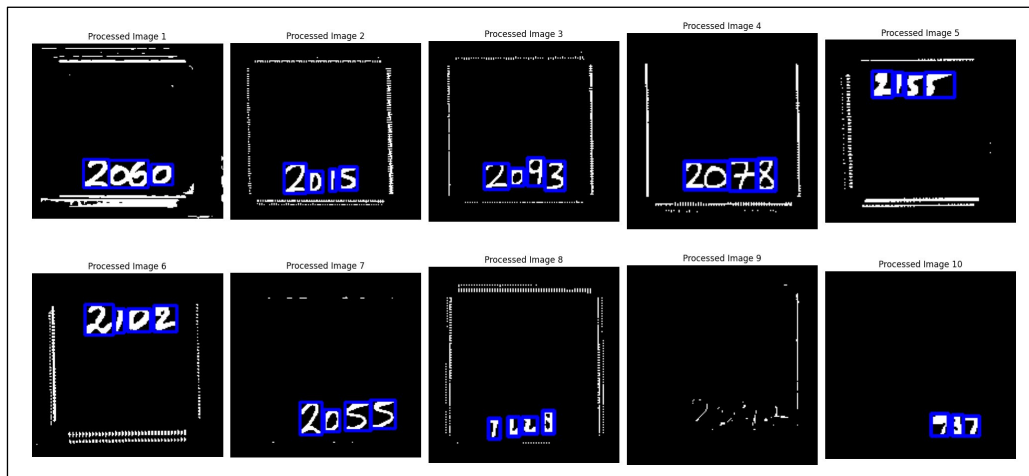


Figure B.6 Chip Tray With Contours Removed Based on Radial Distance, and Vertical and Horizontal Alignment